Sistemas Operativos Multimedia | Práctica 1 | Paso de mensajes

El paso de mensajes mediante *sockets* en sistemas Unix es un mecanismo fundamental de comunicación entre procesos (IPC), permitiendo que procesos que se ejecutan en la misma máquina o en diferentes máquinas conectadas a una red puedan intercambiar datos de manera bidireccional.

A continuación, se presenta una guía detallada con las llamadas al sistema y funciones de biblioteca estándar utilizadas para configurar, establecer y gestionar conexiones de red orientadas a conexión (TCP).

Llamada socket

`int socket(int domain, int type, int protocol)`

La llamada al sistema `socket` crea un extremo de comunicación (un *socket*) y devuelve un descriptor de archivo que lo identifica. Es el punto de partida obligatorio tanto para clientes como para servidores.

- `domain`: Especifica el dominio de comunicación (ej. AF_INET para IPv4, AF_INET6 para IPv6, AF_UNIX para comunicación local).
- `type`: El tipo de comunicación (ej. SOCK_STREAM para conexiones fiables orientadas a flujo como TCP, SOCK_DGRAM para datagramas no orientados a conexión como UDP).
- `protocol`: Especifica un protocolo particular a utilizar con el *socket* (normalmente se indica 0 para que el sistema elija el protocolo por defecto según el tipo).

En este fragmento se invoca `socket` solicitando un *socket* de Internet IPv4 (AF_INET) y de tipo flujo (SOCK_STREAM / TCP). Si la llamada tiene éxito, devuelve un descriptor entero positivo que se utilizará en operaciones posteriores; si falla, devuelve -1.

```c
#include <sys/types.h>
#include <sys/socket.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    if (server_fd == -1) {
        perror("Error al crear el socket");
        exit(EXIT_FAILURE);
    }
    printf("Socket creado correctamente con descriptor: %d\n", server_fd);
    return 0;
}
```

Llamada setsockopt

`int setsockopt(int sockfd, int level, int optname, const void *optval, socklen_t optlen)`

Esta llamada permite manipular las opciones asociadas con un *socket*. Se utiliza comúnmente en los servidores para permitir la reutilización inmediata de direcciones y puertos (SO_REUSEADDR), evitando el error de puerto ocupado al reiniciar la aplicación.

- `sockfd`: El descriptor del *socket* a modificar.
- `level`: El nivel en el que reside la opción (ej. SOL_SOCKET para el nivel general del *socket*).
- `optname`: La opción específica a configurar (ej. SO_REUSEADDR).
- `optval`: Puntero al valor que se desea asignar a la opción.
- `optlen`: El tamaño en bytes de la estructura o valor apuntado por optval.

En el siguiente ejemplo configuramos el *socket* para que permita la reutilización del puerto inmediatamente después de cerrarse, pasando un valor entero 1 a la opción SO_REUSEADDR.

```c
#include <sys/types.h>
#include <sys/socket.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    int opt = 1;

    if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt)) < 0) {
        perror("Error en setsockopt");
        exit(EXIT_FAILURE);
    }
    printf("Opciones de socket configuradas correctamente (SO_REUSEADDR activo).\n");
    return 0;
}
```

Funciones de conversión de direcciones y redes (inet_pton y hton)

Antes de conectar o enlazar un *socket*, es necesario empaquetar las direcciones IP y los puertos en formatos estándar entendibles por el sistema operativo y compatibles con cualquier arquitectura de red.

Función inet_pton

`int inet_pton(int af, const char *src, void *dst)`

Convierte una dirección de red numérica en formato de texto (por ejemplo, "127.0.0.1") a su formato binario en red (estructura in_addr).

- `af`: El dominio de red (AF_INET o AF_INET6).
- `src`: Cadena de caracteres con la dirección IP legible.
- `dst`: Puntero donde se almacenará la dirección en formato binario.

Funciones hton (Host to Network)

`uint16_t htons(uint16_t hostshort)`

`uint32_t htonl(uint32_t hostlong)`

Convierten valores numéricos (como puertos o direcciones IP en formato entero) desde el orden de bytes del procesador local (host) al orden de bytes estándar de la red (network, big-endian).

En el siguiente ejemplo utilizamos `htons` para asegurar que el puerto 8080 esté en el formato correcto de red y `inet_pton` para traducir la dirección IP textual de localhost a su representación binaria dentro de la estructura de dirección.

```c
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    struct sockaddr_in address;
    address.sin_family = AF_INET;
    address.sin_port = htons(8080); // Convierte puerto 8080 a orden de red

    if (inet_pton(AF_INET, "127.0.0.1", &address.sin_addr) <= 0) {
        perror("Dirección inválida o no soportada");
        exit(EXIT_FAILURE);
    }
    printf("Dirección IP y puerto convertidos correctamente a formato de red.\n");
    return 0;
}
```

Llamada bind

`int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen)`

Asigna una dirección local (una combinación de dirección IP y puerto) al *socket* creado. Es un paso obligatorio para los procesos servidores antes de comenzar a escuchar conexiones entrantes.

- `sockfd`: El descriptor del *socket* creado.
- `addr`: Puntero a una estructura que contiene la dirección a asignar (normalmente struct sockaddr_in).

- `addrlen`: El tamaño en bytes de la estructura de dirección proporcionada.

En el siguiente ejemplo vinculamos el *socket* del servidor a cualquier interfaz de red disponible (INADDR_ANY) en el puerto 8080.

```c
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in address;

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(8080);

    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("Error en bind");
        exit(EXIT_FAILURE);
    }
    printf("Socket vinculado al puerto 8080 exitosamente.\n");
    return 0;
}
```

Llamada listen

`int listen(int sockfd, int backlog)`

Marca un *socket* vinculado (bind) como un *socket* pasivo, es decir, un *socket* que se utilizará para aceptar solicitudes de conexión entrantes de clientes.

- `sockfd`: Descriptor del *socket* del servidor.
- `backlog`: El tamaño máximo de la cola de conexiones pendientes que el sistema operativo mantendrá antes de empezar a rechazar nuevas solicitudes.

En el siguiente ejemplo ponemos el *socket* en modo de escucha permitiendo una cola de hasta 3 conexiones pendientes simultáneas.

```c
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
```

```
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in address = {0};
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(8080);

    bind(server_fd, (struct sockaddr *)&address, sizeof(address));

    if (listen(server_fd, 3) < 0) {
        perror("Error en listen");
        exit(EXIT_FAILURE);
    }
    printf("Servidor escuchando conexiones entrantes...\n");
    return 0;
}
```

Llamada accept

`int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen)`

Se utiliza en el servidor para extraer la primera solicitud de conexión de la cola de conexiones pendientes del *socket* en escucha, creando un nuevo descriptor de *socket* conectado específicamente para comunicarse con ese cliente.

- `sockfd`: El descriptor del *socket* en modo escucha.
- `addr`: Puntero a una estructura donde se almacenarán los datos de la dirección del cliente que se conecta (opcional, puede ser NULL).
- `addrlen`: Puntero a un valor que especifica el tamaño de la estructura de dirección del cliente.

El servidor se bloquea en la llamada `accept` hasta que un cliente intenta conectarse. Al establecerse, devuelve un nuevo descriptor dedicado a la comunicación exclusiva con dicho cliente.

```
#include <sys/types.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    int server_fd = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in address = {0};
    int addrlen = sizeof(address);

    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(8080);
```

```
    bind(server_fd, (struct sockaddr *)&address, sizeof(address));
    listen(server_fd, 3);

    printf("Esperando conexión del cliente...\n");
    int new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t*)&addrlen);
    if (new_socket < 0) {
        perror("Error en accept");
        exit(EXIT_FAILURE);
    }
    printf("¡Conexión aceptada! Nuevo socket descriptor: %d\n", new_socket);
    return 0;
}
```

Llamada connect

`int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen)`

Utilizada por el proceso cliente para establecer una conexión activa con un servidor remoto a través de su dirección y puerto.

- `sockfd`: El descriptor del *socket* creado por el cliente.
- `addr`: Puntero a la estructura que contiene la dirección IP y puerto del servidor de destino.
- `addrlen`: Tamaño en bytes de la estructura de dirección del servidor.

En el siguiente ejemplo, el cliente inicializa un *socket* propio y solicita conectarse a la dirección 127.0.0.1 en el puerto 8080 mediante connect.

```
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <stdio.h>
#include <stdlib.h>

int main() {
    int sock = socket(AF_INET, SOCK_STREAM, 0);
    struct sockaddr_in serv_addr = {0};

    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(8080);
    inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("Conexión fallida");
        exit(EXIT_FAILURE);
    }
```

```
    printf("Conectado exitosamente al servidor.\n");
    return 0;
}
```

Llamadas de transmisión de datos (send y write)

Una vez establecida la conexión, los procesos pueden intercambiar información utilizando llamadas de escritura.

Llamada send

`ssize_t send(int sockfd, const void *buf, size_t len, int flags)`

Permite enviar datos a través de un *socket* conectado. El parámetro flags (habitualmente configurado a 0) modifica el comportamiento de la transmisión.

Llamada write

`ssize_t write(int fd, const void *buf, size_t count)`

Trata al *socket* como un descriptor de archivo convencional para enviar datos.

En este ejemplo, ambas funciones escriben un búfer de texto directamente en el canal de comunicación del *socket* para que el extremo receptor pueda procesarlo.

```
#include <sys/socket.h>
#include <unistd.h>
#include <stdio.h>

void enviar_datos(int socket_fd) {
    const char *mensaje = "¡Hola desde el cliente!";

    // Usando send
    ssize_t bytes_enviados = send(socket_fd, mensaje, strlen(mensaje), 0);

    // Alternativa usando write
    // ssize_t bytes_enviados = write(socket_fd, mensaje, strlen(mensaje));

    if (bytes_enviados < 0) {
        perror("Error al enviar datos");
    } else {
        printf("Se enviaron %zd bytes.\n", bytes_enviados);
    }
}
```

Llamadas de recepción de datos (recv y read)

Permiten leer los mensajes entrantes enviados por el proceso remoto a través del descriptor de *socket*.

Llamada recv

`ssize_t recv(int sockfd, void *buf, size_t len, int flags)`

Lee datos de un *socket* conectado, permitiendo especificar opciones mediante flags.

Llamada read

`ssize_t read(int fd, void *buf, size_t count`

Lee datos desde un descriptor de archivo genérico, incluyendo *sockets*.

La función `recv` detiene la ejecución del proceso de forma bloqueante hasta que llegan datos a través del *socket*, almacenándolos en un búfer local para su lectura.

```c
#include <sys/socket.h>
#include <unistd.h>
#include <stdio.h>

void recibir_datos(int socket_fd) {
    char buffer[1024] = {0};

    ssize_t bytes_leidos = recv(socket_fd, buffer, sizeof(buffer) - 1, 0);

    if (bytes_leidos < 0) {
        perror("Error al recibir datos");
    } else if (bytes_leidos == 0) {
        printf("El extremo remoto cerró la conexión.\n");
    } else {
        printf("Mensaje recibido: %s\n", buffer);
    }
}
```

Llamada close

`int close(int fd)`

Libera el descriptor de archivo asociado al *socket* y termina formalmente la conexión de red, liberando los recursos reservados en el sistema operativo.

- `fd`: El descriptor de archivo del *socket* que se desea cerrar.

Al finalizar la transmisión de mensajes, tanto el cliente como el servidor invocan close sobre sus respectivos descriptores de *socket* para cerrar la conexión de manera limpia.

```c
#include <unistd.h>
#include <stdio.h>
```

```c
void cerrar_conexion(int socket_fd) {
    if (close(socket_fd) < 0) {
        perror("Error al cerrar el socket");
    } else {
        printf("Socket cerrado correctamente y recursos liberados.\n");
    }
}
```